

EXPLICACIÓN PROYECTO .NET FASE 1

Este documento explica las pruebas de funcionalidad realizadas sobre el Sistema de Gestión de Expedientes (SGE), el cuál desarrollamos bajo los principios de Arquitectura Limpia y tácticas de Domain-Driven Design (DDD). El proyecto SGE.Consola cumple la función crítica de punto de entrada. En este nivel, se encargará de instanciar las implementaciones concretas de la capa de Infraestructura (como los repositorios de persistencia en archivos de texto y el servicio de autorización provisional) para luego poder inyectarlas como dependencias en los casos de uso de la capa de Aplicación.

1. Prueba del flujo estándar (camino feliz):

En esta prueba, simulamos una creación exitosa de expedientes con datos válidos, su listado y la incorporación de un trámite que aplica la lógica de actualización de estado.

```
try
{
    Console.WriteLine("=== CREANDO EXPEDIENTE ===");

    // DTO para crear expediente
    var expedienteRequest = new AgregarExpedienteRequest("Expediente de prueba", Guid.NewGuid());

    // Ejecuta caso de uso
    agregarExpedienteUseCase.Ejecutar(expedienteRequest);

    Console.WriteLine("Expediente creado correctamente");

    Console.WriteLine("\n=== LISTANDO EXPEDIENTES ===");

    // Uso el caso de uso para obtener DTOs, no entidades
    var expedientesResponse = listarExpedientesUseCase.Ejecutar(new ListarTodosLosExpedientesRequest());

    foreach (var expediente in expedientesResponse.Expedientes)
    {
        Console.WriteLine($"{expediente.Id} | {expediente.Caratula} | {expediente.Estado}");
    }
}
```

```
Console.WriteLine("\n=== AGREGANDO TRÁMITE ===");

// Toma el primer expediente
var primerExpediente = expedientesResponse.Expedientes.First();

// DTO para trámite
var tramiteRequest = new AgregarTramiteRequest(primerExpediente.Id,
EtiquetaTramite.Resolucion, "Se resolvió el expediente", Guid.NewGuid());

// Ejecuta caso de uso
agregarTramiteUseCase.Ejecutar(tramiteRequest);

Console.WriteLine("Trámite agregado correctamente");

Console.WriteLine("\n=== LISTANDO TRÁMITES DEL EXPEDIENTE ===");

var tramitesResponse = listarTramitesUseCase.Ejecutar(new
ListarTramitesPorExpedienteRequest(primerExpediente.Id));

foreach (var tramite in tramitesResponse.Tramites)
{
    Console.WriteLine($"{tramite.Id} | {tramite.Etiqueta} |
{tramite.Contenido}");
}

// Captura de errores
catch (DominioException ex)
{
    Console.WriteLine($"Error de dominio: {ex.Message}");
}

catch (RepositorioException ex)
{
    Console.WriteLine($"Error de repositorio: {ex.Message}");
}

catch (AutorizacionException ex)
{

```

```

        Console.WriteLine($"Error de autorización: {ex.Message}");
    }

    catch (Exception ex)
    {
        Console.WriteLine($"Error general: {ex.Message}");
    }

```

Salida por consola:

=== CREANDO EXPEDIENTE ===

Expediente creado correctamente

=== LISTANDO EXPEDIENTES ===

c309aa64-6b4e-4ee8-a72e-3f0892206d27 | Expediente de prueba | ConResolucion
 0888cbc1-4376-4ac7-a6f5-d458c07f2906 | Expediente de prueba | RecienIniciado

=== AGREGANDO TRAMITE ===

Trámite agregado correctamente

=== LISTANDO TRAMITES DEL EXPEDIENTE ===

06671787-7548-441d-8929-77978e1549c9 | Resolucion | Se resolvió el expediente
 d6217f99-28e8-470f-a5ee-2587bc0e5d0c | Resolucion | Se resolvió el expediente

2. Prueba reglas del sistema:

En esta prueba, intentamos crear un expediente con una carátula vacía, la cual debería ser rechazada automáticamente y generar una excepción.

```

Console.WriteLine("=== PRUEBA: CARÁTULA INVÁLIDA ===");
try
{
    // Intentamos enviar un string vacío al request
    var requestMalo = new AgregarExpedienteRequest("", Guid.NewGuid());
    agregarExpedienteUseCase.Ejecutar(requestMalo);
}
catch (DominioException ex)
{
    // El sistema atrapa la violación de la regla de negocio
    Console.WriteLine($"Error de Negocio: {ex.Message}");
}

```

```

catch (AutorizacionException ex)
{
    Console.WriteLine($"Error de seguridad: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"[Error Inesperado]: {ex.Message}");
}

```

Salida por consola:

=== PRUEBA: CARÁTULA INVALIDA ===

Error de Negocio: La carátula no puede estar vacía o ser nula

3. Prueba entidad no encontrada:

En esta prueba, intentamos realizar una operación sobre un Id de expediente que no existe,

```

Console.WriteLine("\n=== PRUEBA: ENTIDAD NO ENCONTRADA ===");
try
{
    // Usamos un Guid cualquiera que no esté en nuestros archivos .txt
    Guid idInexistente = Guid.NewGuid();
    var reqTramite = new AgregarTramiteRequest(idInexistente,
    EtiquetaTramite.Despacho, "Contenido", Guid.NewGuid());

    agregarTramiteUseCase.Ejecutar(reqTramite);
}
catch (EntidadNoEncontradaException ex)
{
    Console.WriteLine($"[Error de Aplicación]: {ex.Message}");
}
catch (AutorizacionException ex)
{
    Console.WriteLine($"Error de seguridad: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"[Error Inesperado]: {ex.Message}");
}

```

Salida por consola:

=== PRUEBA: ENTIDAD NO ENCONTRADA ===

[Error de Aplicación]: No existe el expediente con ID ccc12a0b-5308-4cc0-9547-9d2f45fc6905

3. Prueba de trámite con contenido inválido:

En esta prueba, intentamos asociar un trámite a un expediente válido, pero mandando un texto vacío.

```
Console.WriteLine("\n=== PRUEBA: CONTENIDO DE TRÁMITE VACÍO ===");
try
{
    // obtenemos un id de expediente que sí existe para que pase la
    primera barrera
    var expedienteExistente = listarExpedientesUseCase.Ejecutar(new
ListarTodosLosExpedientesRequest()).Expedientes.First();

    // intentamos crear un trámite con contenido inválido (string vacío)
    var requestTramiteMalo = new AgregarTramiteRequest(
        expedienteExistente.Id,
        EtiquetaTramite.EscritoPresentado,
        "", // <--- Contenido inválido
        Guid.NewGuid()
    );

    agregarTramiteUseCase.Ejecutar(requestTramiteMalo);
}
catch (DominioException ex)
{
    // El Value Object 'ContenidoTramite' captura el error al instanciarse
    Console.WriteLine($"Error de Negocio: {ex.Message}");
}
catch (AutorizacionException ex)
{
    Console.WriteLine($"Error de seguridad: {ex.Message}");
}
catch (Exception ex)
{

```

```
Console.WriteLine($"[Error Inesperado]: {ex.Message}");
}
```

Salida por consola:

=== PRUEBA: CONTENIDO DE TRÁMITE VACÍO ===

Error de Negocio: El contenido del trámite no puede estar vacío

4. Prueba de error de autorización:

En esta prueba, cambiamos manualmente el valor del método PoseeElPermiso a false (luego lo dejamos en true nuevamente, es solo para ver como funciona con el false) e intentamos realizar un procedimiento que necesite permiso.

```
Console.WriteLine("\n=== PRUEBA: USUARIO NO AUTORIZADO ===");
try
{
    // Intentamos crear un expediente.
    // Como el servicio ahora devuelve 'false', el Use Case lanzará la
    excepción.
    var requestCualquiera = new AgregarExpedienteRequest("Expediente de
Prueba", Guid.NewGuid());
    agregarExpedienteUseCase.Ejecutar(requestCualquiera);
}
catch (AutorizacionException ex)
{
    // Capturamos y mostramos el mensaje de error de seguridad
    Console.WriteLine($"Error de Seguridad: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"[Error Inesperado]: {ex.Message}");
}
```

Salida por consola:

=== PRUEBA: USUARIO NO AUTORIZADO ===

Error de Seguridad: Usuario no autorizado para crear expedientes

(Básicamente, genera ese error en todas las pruebas anteriores al ejecutarlo, pero al mostrar solo esta prueba adjuntamos la salida por consola de ese único pedazo de código).